

Refactoring

<https://refactoring.guru/>

La principale raison du remaniement (refactor) est de lutter contre la dette technique. Elle transforme le désordre en code propre et en conception simple.

Bien ! Mais qu'est-ce que le code propre, de toute façon ? Voici quelques-unes de ses caractéristiques :

Le **code propre** est évident pour les autres programmeurs.

Et je ne parle pas d'algorithmes super sophistiqués. Des noms de variables peu clairs, des classes et des méthodes surchargées, des nombres magiques - appelez-les comme vous voulez - tout cela rend le code brouillon et difficile à comprendre.

Le **code propre** ne contient pas de redondance.

Chaque fois que vous devez apporter une modification à un code en double, vous devez vous rappeler d'apporter la même modification à chaque instance. Cela augmente la charge cognitive et ralentit le progrès.

Le code propre contient un nombre minimal de classes et d'autres éléments mobiles.

Moins de code signifie moins de choses à retenir. Moins de code signifie moins de maintenance. Moins de code signifie moins de bogues. Le code est une responsabilité, gardez-le court et simple.

Le code propre réussit tous les tests.

Vous savez que votre code est sale lorsque seuls 95 % de vos tests ont réussi. Vous savez que vous êtes dans de beaux draps lorsque votre couverture de tests est de 0 %.

Le code propre est plus facile et moins cher à entretenir !”

Dette technique

Tout le monde fait de son mieux pour écrire un code excellent dès le départ. Il n'y a probablement pas de programmeur qui écrive intentionnellement un code non propre au détriment du projet. Mais à quel moment le code propre devient-il non propre ?

La métaphore de la « **dettes techniques** » en ce qui concerne le code non propre a été initialement suggérée par Ward Cunningham.

Si vous obtenez un prêt auprès d'une banque, cela vous permet d'effectuer des achats plus rapidement. Vous payez un supplément pour accélérer le processus - vous ne remboursez pas seulement le capital, mais également les intérêts supplémentaires sur le prêt. Inutile de dire que vous pouvez même accumuler tellement d'intérêts que le montant des intérêts dépasse l'ensemble de vos revenus, rendant le remboursement complet impossible.

La même chose peut se produire avec le code. Vous pouvez temporairement accélérer le développement sans écrire de tests pour de nouvelles fonctionnalités, mais cela ralentira progressivement votre progression chaque jour jusqu'à ce que vous remboursiez finalement la dette en écrivant des tests.

Causes de la dette technique

Pression commerciale

Parfois, les circonstances commerciales peuvent vous contraindre à déployer des fonctionnalités avant qu'elles ne soient complètement terminées. Dans ce cas, des correctifs et des bricolages apparaissent dans le code pour masquer les parties inachevées du projet.

Manque de compréhension des conséquences de la dette technique

Parfois, votre employeur peut ne pas comprendre que la dette technique a des « intérêts » en ce sens qu'elle ralentit le rythme de développement à mesure que la dette s'accumule. Cela peut rendre trop difficile de consacrer du temps au remaniement, car la direction ne voit pas sa valeur.

Échec à combattre la stricte cohérence des composants

Cela se produit lorsque le projet ressemble davantage à un monolithe qu'à un produit composé de modules individuels. Dans ce cas, toute modification apportée à une partie du projet affectera les autres. Le développement en équipe est compliqué car il est difficile d'isoler le travail des membres individuels.

Manque de tests

Le manque de rétroaction immédiate encourage des contournements ou des bricolages rapides mais risqués. Dans les pires cas, ces modifications sont mises en œuvre et déployées directement en production sans aucun test préalable. Les conséquences peuvent être catastrophiques. Par exemple, une correction mineure en apparence peut envoyer un e-mail de test inhabituel à des milliers de clients, voire pire, vider ou corrompre une base de données entière.

Manque de documentation

Cela ralentit l'intégration de nouvelles personnes dans le projet et peut bloquer le développement si des personnes clés quittent le projet.

Causes de la dette technique (suite...)

Manque d'interaction entre les membres de l'équipe

Si la base de connaissances n'est pas partagée dans toute l'entreprise, les personnes finissent par travailler avec une compréhension dépassée des processus et des informations sur le projet. Cette situation peut s'aggraver lorsque les développeurs juniors sont mal formés par leurs mentors.

Développement simultané à long terme sur plusieurs branches

Cela peut entraîner l'accumulation de dette technique, qui augmente lorsque les modifications sont fusionnées. Plus il y a de modifications effectuées en isolation, plus la dette technique totale augmente.

Remaniement retardé

Les besoins du projet changent constamment et à un moment donné, il peut devenir évident que certaines parties du code sont obsolètes, devenues lourdes et doivent être repensées pour répondre aux nouvelles exigences.

D'un autre côté, les programmeurs du projet écrivent chaque jour un nouveau code qui fonctionne avec les parties obsolètes. Par conséquent, plus le remaniement est retardé, plus il faudra retravailler le code dépendant à l'avenir.

Manque de surveillance de la conformité

Cela se produit lorsque tout le monde travaillant sur le projet écrit le code comme il le souhaite (c'est-à-dire de la même manière qu'il a écrit le dernier projet). (SonarQube?, Bufix, Linter)

Incompétence

Cela se produit lorsque le développeur ne sait tout simplement pas comment écrire un code décent.

Quand effectuer du ‘remaniement’ (refactor)

La règle des trois

Lorsque vous faites quelque chose pour la première fois, contentez-vous de le faire.

Lorsque vous faites quelque chose de similaire pour la deuxième fois, grimacez à l'idée de devoir répéter, mais faites-le quand même.

Lorsque vous faites quelque chose pour la troisième fois, commencez à remanier.

Lors de l'ajout d'une fonctionnalité

Le remaniement vous aide à comprendre le code des autres. Si vous devez travailler avec un code malpropre écrit par quelqu'un d'autre, essayez d'abord de le remanier. Le code propre est beaucoup plus facile à comprendre. Vous l'améliorerez non seulement pour vous-même, mais aussi pour ceux qui l'utiliseront après vous.

Le remaniement facilite l'ajout de nouvelles fonctionnalités. Il est beaucoup plus facile d'apporter des modifications dans un code propre.

Lors de la correction d'un bug

Les bogues dans le code se comportent comme ceux de la vie réelle : ils vivent dans les endroits les plus sombres et les plus sales du code. Nettoyez votre code et les erreurs se découvriront pratiquement d'elles-mêmes.

Les gestionnaires apprécient le remaniement proactif car il élimine le besoin de tâches de remaniement spéciales ultérieurement. Des patrons satisfaits rendent les programmeurs heureux !

Pendant une revue de code

La revue de code peut être la dernière chance de nettoyer le code avant qu'il ne devienne disponible au public.

Il est préférable de réaliser de telles revues en binôme avec l'auteur. De cette façon, vous pouvez résoudre rapidement les problèmes simples et évaluer le temps nécessaire pour résoudre les problèmes plus difficiles.

Comment remanier (refactor)

Le remaniement doit être effectué sous forme d'une série de petits changements, chacun améliorant légèrement le code existant tout en laissant le programme en état de fonctionnement.

Liste de contrôle du remaniement effectué correctement (checklist)

Le code devrait devenir plus propre.

Si le code reste aussi malpropre après le remaniement... eh bien, je suis désolé, mais vous venez de perdre une heure de votre vie. Essayez de comprendre pourquoi cela s'est produit.

Cela arrive fréquemment lorsque vous abandonnez le remaniement par petits changements et que vous mélangez une série de remaniements en un seul grand changement. Il est donc très facile de perdre la tête, surtout si vous avez une limite de temps.

Mais cela peut également se produire lorsque vous travaillez avec un code extrêmement bâclé. Quoi que vous amélioriez, le code dans son ensemble reste un désastre.

Dans ce cas, il est utile de réfléchir à la réécriture complète de certaines parties du code. Mais avant cela, vous devriez avoir écrit des tests et réservé un bon moment. Sinon, vous obtiendrez les résultats dont nous avons parlé au premier paragraphe.

Les nouvelles fonctionnalités ne doivent pas être créées lors du remaniement.

Ne mélangez pas le remaniement et le développement direct de nouvelles fonctionnalités. Essayez de séparer ces processus, au moins dans le cadre de commits individuels.

Tous les tests existants doivent réussir après le remaniement.

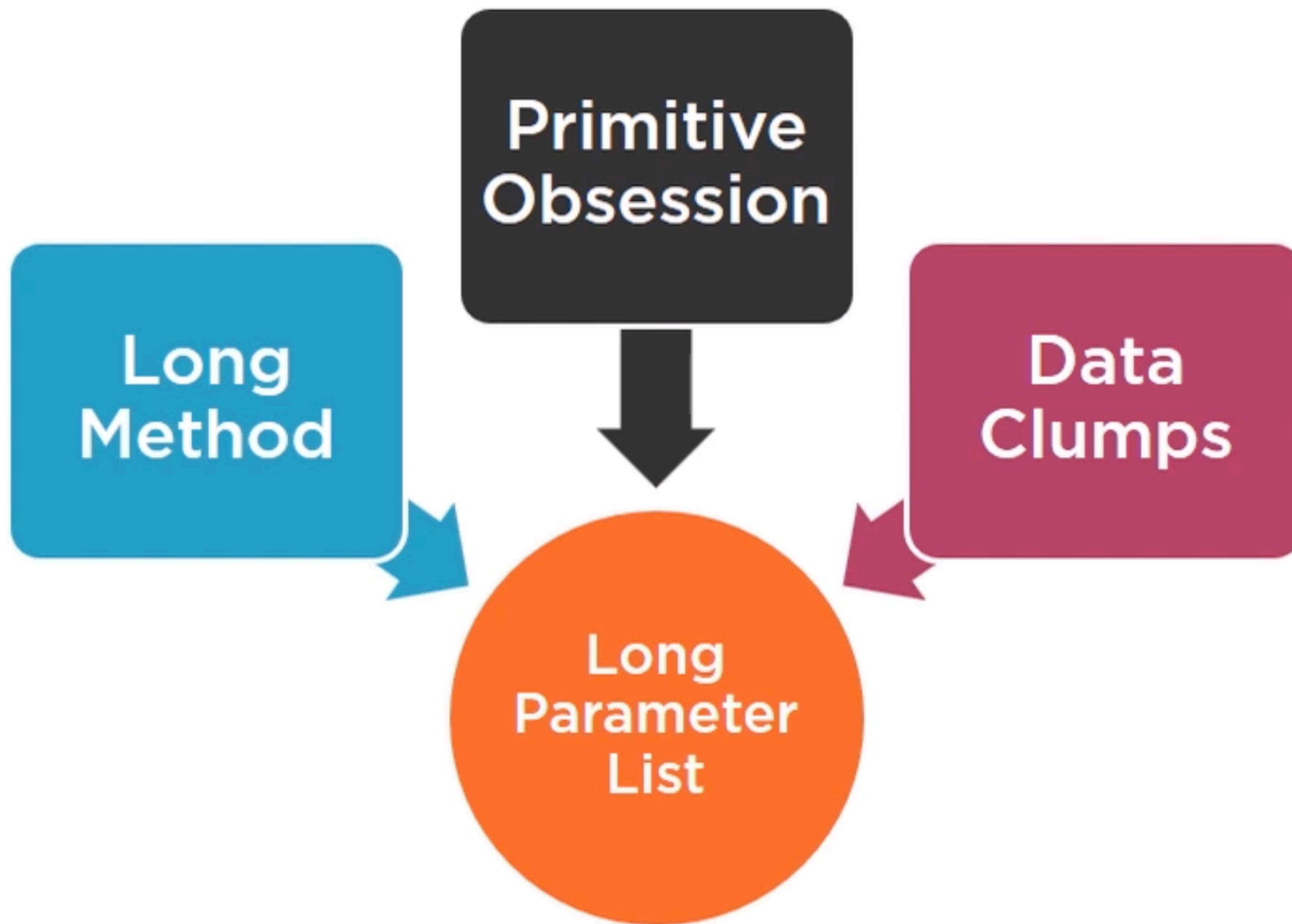
Il y a deux cas où les tests peuvent échouer après le remaniement

Vous avez commis une erreur lors du remaniement. Celui-ci est évident : allez-y et corrigez l'erreur.

Vos tests étaient trop détaillés. Par exemple, vous testiez des méthodes privées de classes.

Dans ce cas, les tests sont en cause. Vous pouvez soit remanier les tests eux-mêmes, soit rédiger un tout nouvel ensemble de tests de plus haut niveau. Une excellente manière d'éviter ce genre de situation est de rédiger des tests de style BDD (Behavior-Driven Development).

Code smells



```
double calculateTotal(List<Item> items, String voucher,  
String membership, String address);
```



Customer { }

```
double calculateTotal(List<Item> items, String voucher,  
String membership, String address);
```

The diagram illustrates the relationship between the `calculateTotal` method and the `Order` and `Customer` classes. A green box highlights the parameters of the `calculateTotal` method: `List<Item> items`, `String voucher`, `String membership`, and `String address`. A green arrow points from the top of this box to the `Order { }` class, indicating that the method is associated with the `Order` class. Another green arrow points from the bottom of the box to the `Customer { }` class, indicating that the method is also associated with the `Customer` class.

Order { }

Customer { }

Preserve Whole Object

```
String a = customer.getA();  
String b = customer.getB();  
double calculateTotal(a, b){ }
```

```
double calculateTotal(customer){  
    String a = customer.getA();  
    String b = customer.getB();  
}
```

Introduce Parameter Object

```
class Order {  
    String a;  
    String b;  
}  
  
double calculateTotal(a, b){ }  
  
Order order = new Order();  
double calculateTotal(order){ }
```

```
sender.sendEmailTo("Mr.", "John", "Doe");
```

```
system.bookRoom(startTime, endTime);
```

```
system.bookReturnFlight(there, back);
```

```
manager.setDeliveryTimeWindow(from, to);
```

Data Clumps - problèmes

Major cause of long parameter lists

Causes code duplication

Voir Main1 - App - Ligne 41

Autre exemple de 'primitive obsession'

```
package com.onlineshop.entities;

public enum Membership {
    BRONZE, SILVER, GOLD
}

36 @
37
38
39
40 public double applyVoucher(String voucher, double price) {
41     // check if voucher is valid
42     if (isValidVoucher(voucher)) {
43         price = BigDecimal.valueOf(price * 0.95).setScale(2,
44             RoundingMode.HALF_EVEN).doubleValue();
45     } else {
46         // invalid voucher
47     }
48 }
```

Voir Main4 - CheckoutHandlerTest - Ligne 31


```
package com.onlineshop.entities;
```

```
import java.time.LocalDate;
```

```
public class Voucher {
```

```
    String code;
```

```
    LocalDate startDate;
```

```
    LocalDate expiryDate;
```

```
}
```

```
36 @
```

```
37
```

```
38
```

```
39
```

```
40
```

```
41
```

```
42
```

```
43
```

```
44
```

```
45
```

```
46
```

```
47
```

```
48
```

```
49
```

```
50 @
```

```
51
```

```
52
```

```
53
```

```
54 @
```

```
55
```

```
56
```

```
private boolean isEligibleForFreeDelivery(String membership) {
```

```
    return membership.equalsIgnoreCase("GOLD");
```

```
}
```

```
public double applyVoucher(String voucher, double price) {
```

```
    // check if voucher is valid
```

```
    if (isValidVoucher(voucher)) {
```

```
        price = BigDecimal.valueOf(price * 0.95).setScale(2,  
RoundingMode.HALF_EVEN).doubleValue();
```

```
    } else {
```

```
        System.out.println("Voucher invalid");
```

```
}
```

```
    return price;
```

```
}
```

```
private boolean isUsAddress(String address) {
```

```
    return address.contains("US");
```

```
}
```

```
public double calculateTotal(Order order) {
```

```
    double baseTotal = sumItemPrices(order.getItems());
```

Bloaters

Large Class

How to Fix Large Class



Extract Class

(Split into several smaller classes)

Voir Main1 - DiscountManager - DeliveryManager

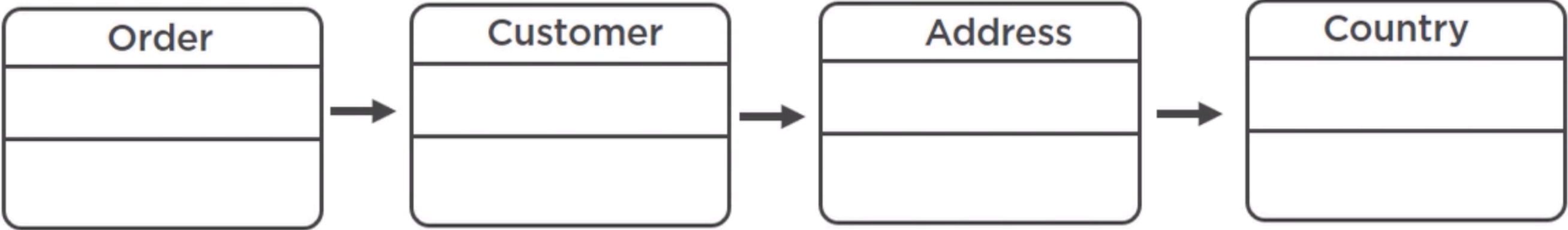
Object Oriented Abusers

- Conditionnal complexity
- Refused Bequest
- Alternative Classes with different interfaces

Évolution du projet

Voir Main2 - Customer

- Nouvelles variables d'instance + ajout d'âge
- Nouveaux constructeurs
- Ajout de pays



Complex Conditionals

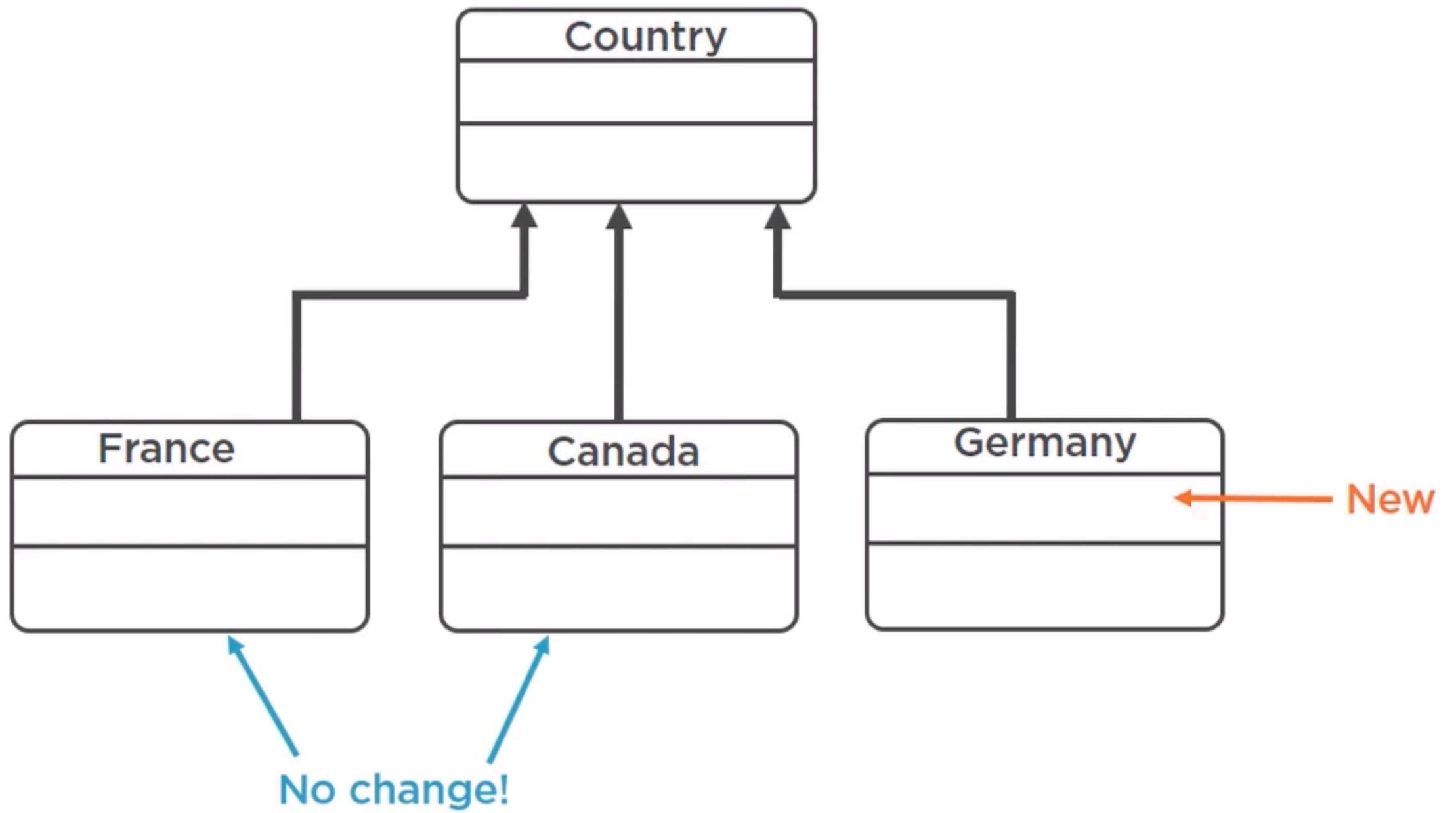
Voir Main3 - App + Order ligne 39

- Missing domain objects
- Not using polymorphism
- Not using inheritance

Comment arranger ça

- À l'aide du polymorphisme
- 1- Voir main3 - Country classes
- 2- Voir main3 - Order ligne 92
- 3- Voir main3 - App
- L'information qui peut changer est maintenant encapsulée dans les bonnes classes, ie Countries.

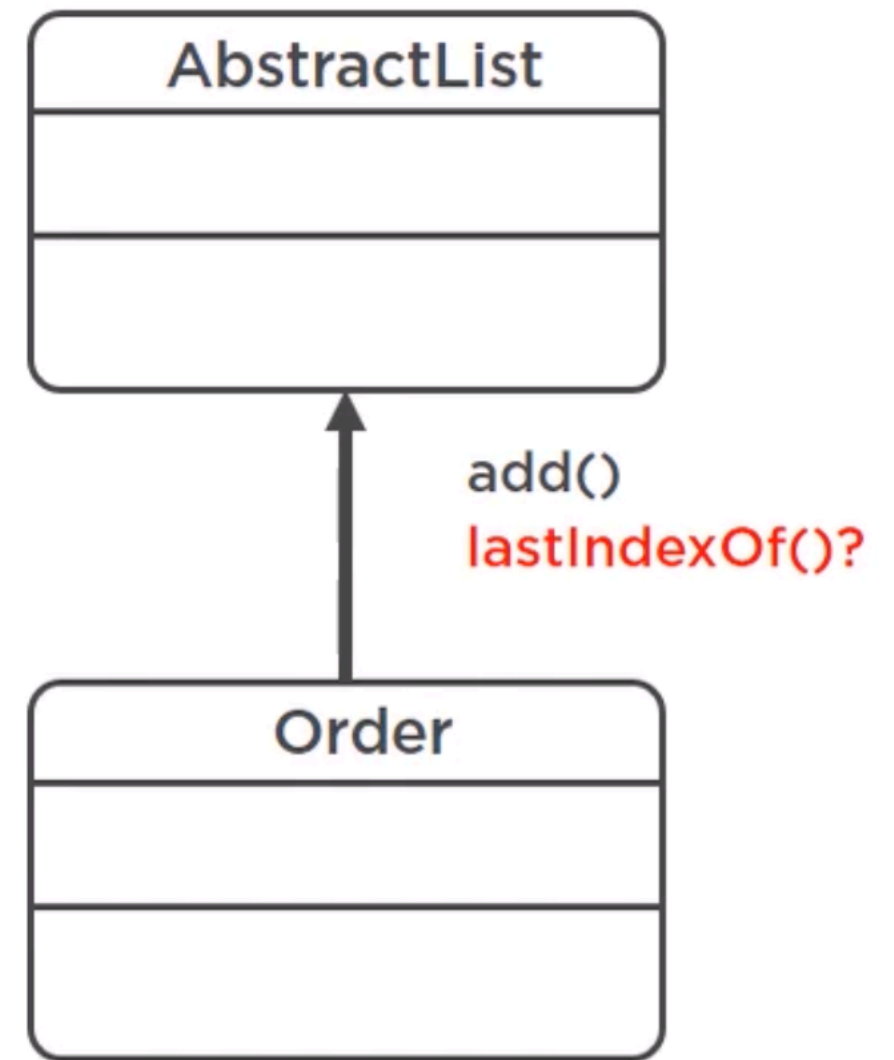
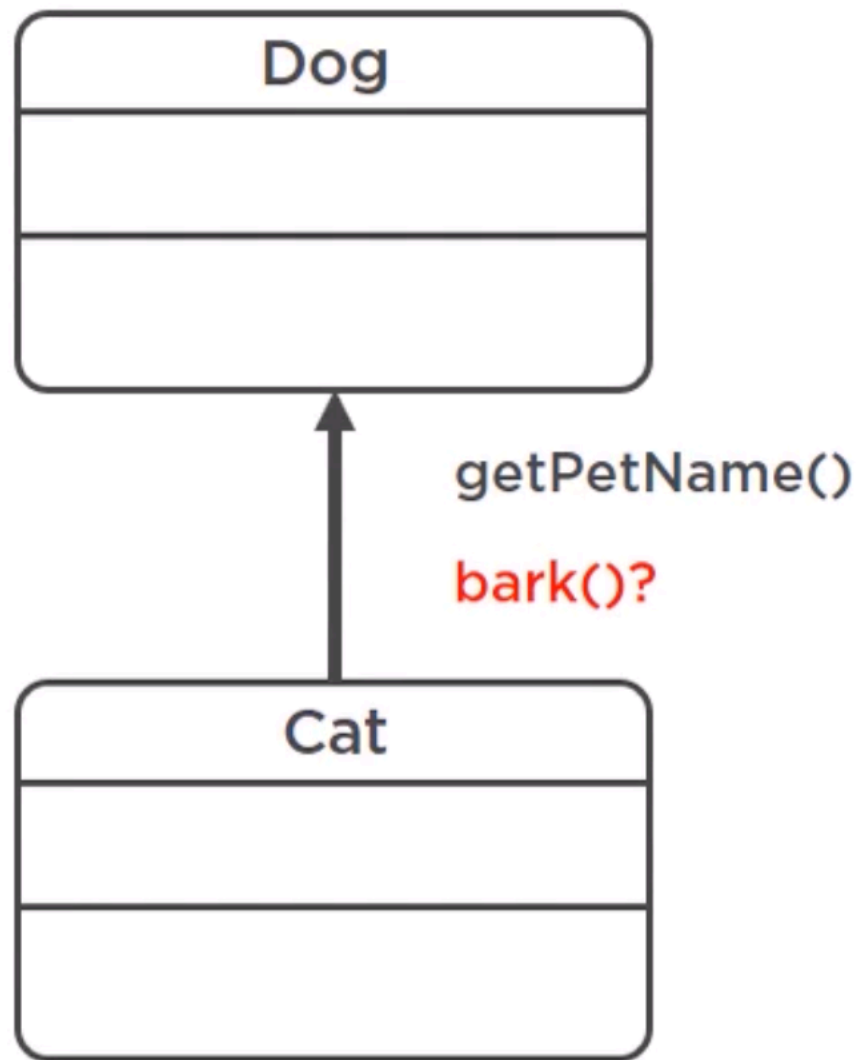
Avantage



Refused Bequest

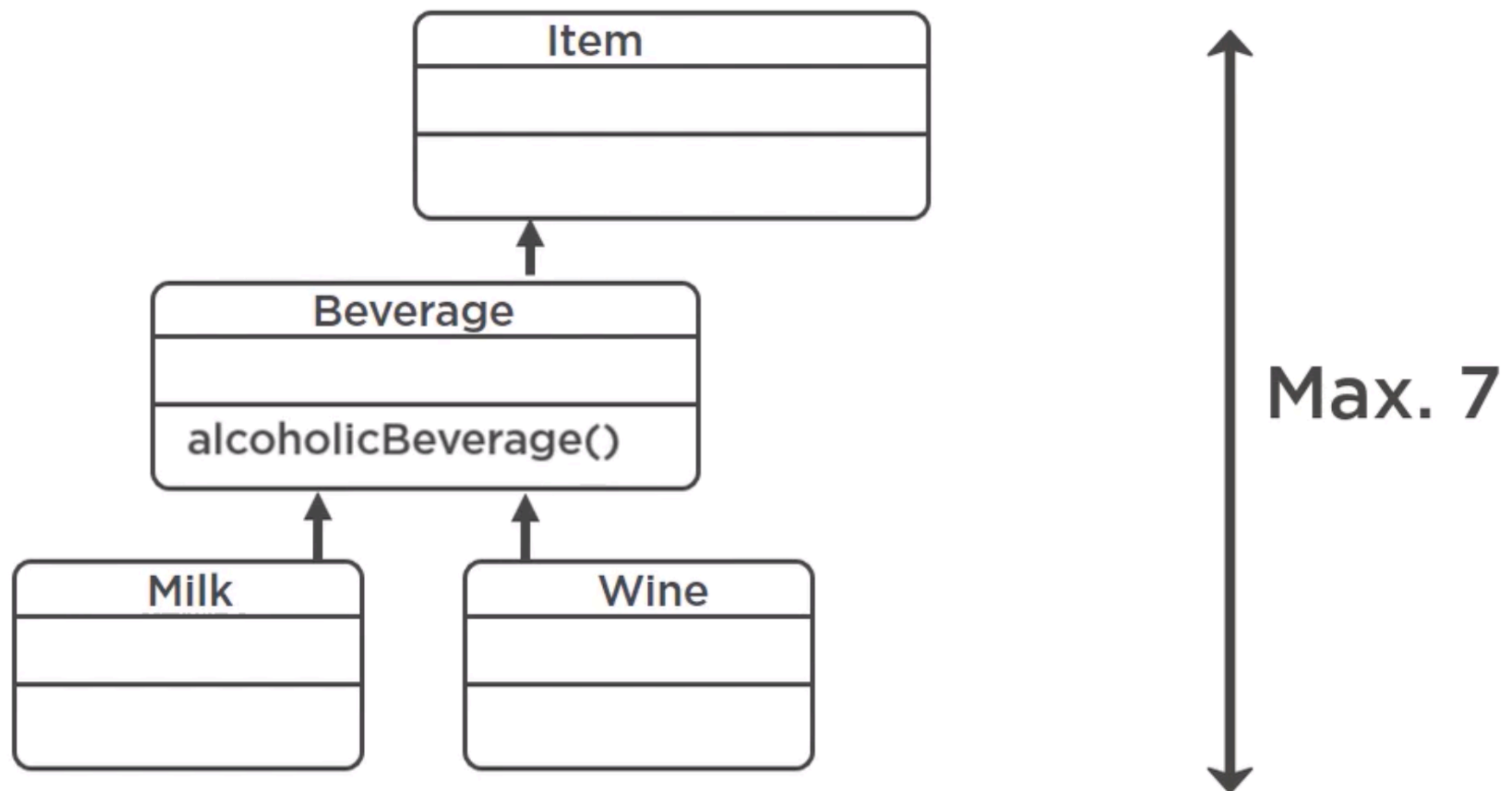
- Un Bequest est un acte de donné à un autre ce que l'on a (exemple, un testament)
- Un Refused Bequest est l'acte de refuser cet acte
- En OO, cela se traduit par une classe qui hérite de méthodes dont elle n'a pas besoin. Et ça, c'est un problème

Unwanted Inheritance



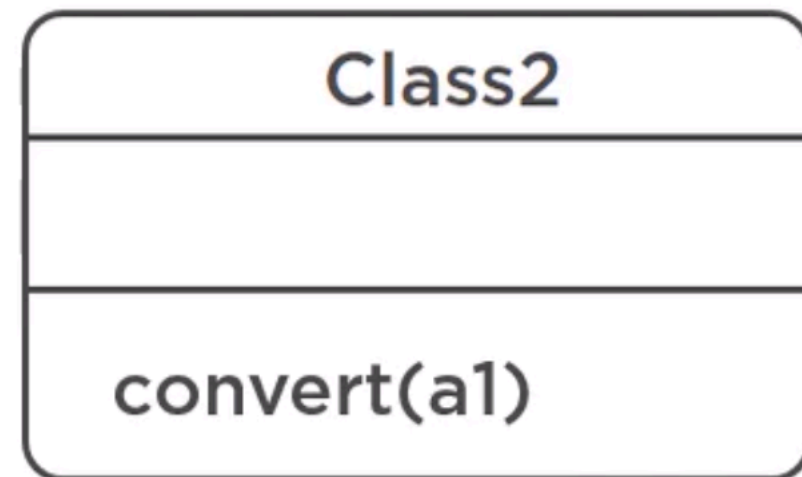
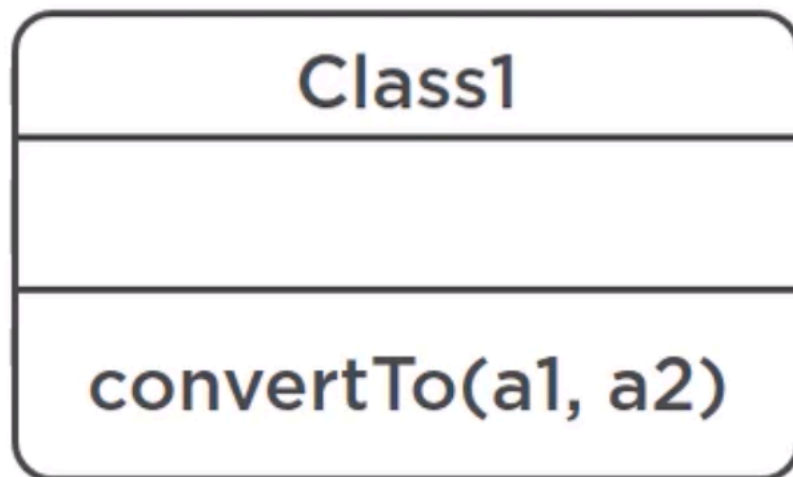
Solution au refused bequest

Create Class and Push Down



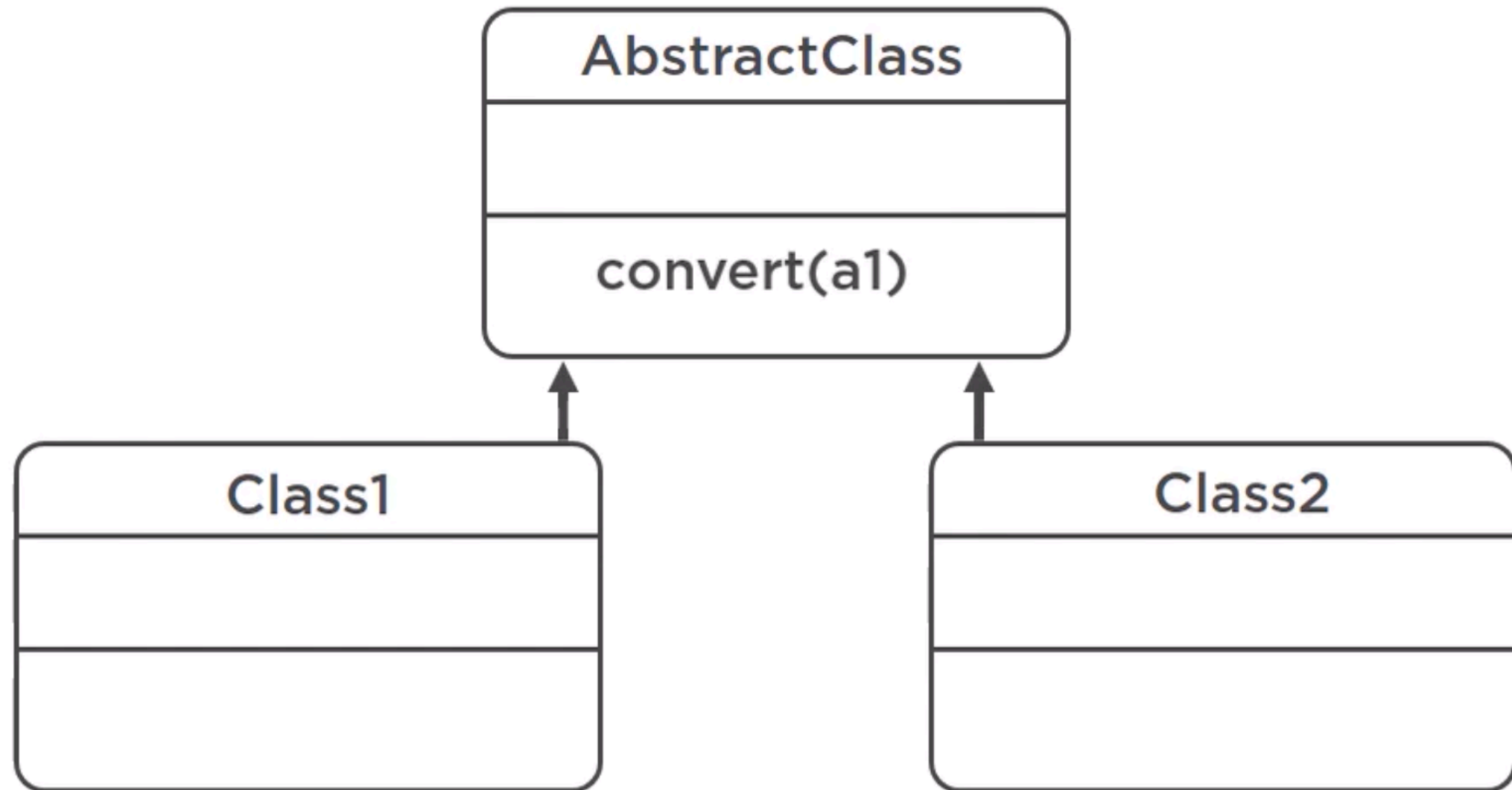
Alternative classes with different interfaces

Alternative Classes with Similar Methods



Une solution possible

Alternative Classes with Similar Methods



Une autre solution

- Des fois c'est impossible d'extraire une interface commune. Une autre solution consiste alors à n'utiliser qu'une seule des 2 classes.